Mattermost Migration Documentation

Pre-requisites:

- Virtual server (VPS) with Ubuntu 24.04 installed
- Root privileges or an account with sudo
- Windows 10 or later or Windows Server 2016 or later
- An Intel Pentium 4 processor or later that is SSE3 capable

The Mattermost migration was completed by installing it, then exporting data from Skype and creating scripts to manipulate and post that data to Mattermost. The Mattermost installation was difficult, however the steps that were taken and the fixes to the errors faced can be seen in the document “20250415a Clockwork (Code Crusaders JN) Running Mattermost Locally Guide v1a”. Data from Skype was exported for the majority of the development of the scripts from accounts that contained only 1 short conversation, for testing purposes. This was done by accessing https://go.skype.com/export and logging in with the account whose data is desired to be exported.

There are 2 scripts to enable the successful migration of Skype conversations to Mattermost: “messages_corrector.py” and “skype_to_mattermost.py”. To initiate this process, a directory is created including these 2 scripts and the exported Skype data zip file is extracted to the created directory’s location. Then the directory is navigated to in command prompt and the “messages_corrector.py” script is ran using “python messages_corrector.py”.

The “messages_corrector.py” script is necessary as the exported messages.json file from Skype is initially ordered from the newest message at the top of the file, to the oldest message at the bottom of the file. This means that should the messages be migrated to Mattermost in that order, the messages would seem to be in reverse order. As a result, this script is first ran to reverse the order of messages in the messages.json file, outputting the result to a respective “messages_reversed.json” file, to ensure the correct order of migrated messages in Mattermost. This is performed simply by reading the file, storing all the conversations as elements in an array, and using the “reverse()” method on it.

```
for convo in data['conversations']:

    convo['MessageList'].reverse()
```

The Skype to Mattermost migration script “skype_to_mattermost.py” is then ran in command prompt using “python skype_to_mattermost.py”, and performs the migration using the previously

created “messages_reversed.json” file. To even begin running this script on Mattermost, several details of the Mattermost server need to be obtained and included in it. This includes:

URL: http://localhost:8065/

Authentication Token: ghamphdmk3bpbbwnwt6x8t7kno

Team ID: 1s3j5b8udt8zxfzy7djeentw6a

Channel ID for Migration Updates Channel: 7hbzag8117f69n6mhpqg3b1rco

media_folder: C:\Users\James\Desktop\Skype Migration\Scripts\media

Then these details are used to initialize variables necessary to authenticate and use the script:

```
# Mattermost API Setup

mattermost_post_url = 'http://localhost:8065/api/v4/posts'

mattermost_file_url = 'http://localhost:8065/api/v4/files'

mattermost_channel_url = 'http://localhost:8065/api/v4/channels'

auth_token = 'ghamphdmk3bpbbwnwt6x8t7kno'  # Your token

team_id = '1s3j5b8udt8zxfzy7djeentw6a'  # ❗Insert your Mattermost team ID here


headers = {

   'Authorization': f'Bearer {auth_token}',

}


media_folder = r'C:\Users\James\Desktop\Skype Migration\Scripts\media'


# Channel ID for "Migration Documentation" where updates will be posted

migration_channel_id = "7hbzag8117f69n6mhpqg3b1rco"  # Use your channel ID
```

Following this, functions are created to create a channel for each conversation, get all the necessary pieces of data from each message in the “messages_reversed.json” file and put them together in a readable way for users accessing Mattermost, and post migration updates in the respective channel. These functions are all called by a for loop that runs for each converastion in the “messages_reversed.py” file. The first one being the “create_channel” function that gets the name from the current Skype conversation being handled, and creates a new channel in Mattermost using the authentication variables with the same name as the Skype conversation.

```
def create_channel(name, display_name):

  safe_name = re.sub(r'[^a-z0-9\-]', '-', name.lower())  # Mattermost channel names must be lowercase and hyphenated

  payload = {

    "team_id": team_id,

    "name": safe_name,

    "display_name": display_name,

    "type": "P"  # "P" = private channel; use "O" for public

  }

  response = requests.post(mattermost_channel_url, headers={**headers, 'Content-Type': 'application/json'}, json=payload)

  if response.status_code == 201:

    return response.json()['id']

  elif response.status_code == 400 and "A channel with that name already exists" in response.text:

    print(f"⚠️ Channel '{safe_name}' already exists. Please delete manually or handle in script.")

    return None

  else:

    print(f"❌ Failed to create channel: {response.status_code} {response.text}")

    return None
```

Then the for loop runs another for loop for each message in the current conversation, and runs the "get_user_display_name" function to get the name of the user who sent the current message, or if a name cannot be found default back to the user's ID:

```
# Map user IDs to display names dynamically

def get_user_display_name(user_id, msg):

  display_name = msg.get('displayName', None)



  if display_name:

    return display_name

  else:

    return user_id
```

Then the rest of the details are obtained from the specific message in the file, with the date and time formatted to be readable, and everything is then packaged and posted to the Mattermost server. An extra conditional is used as well, to handle multimedia, where should a message contain it, the script will pull the desired image from the /media directory and post it to Mattermost with the message "[user] uploaded a file:"

```
conversation = []

  for msg in convo.get('MessageList', []):

    sender_id = msg.get('from')

    display_name = get_user_display_name(sender_id, msg)

    print("actual displayname gotten")

    print(display_name)



    # Only process messages from valid users

    if sender_id:

      conversation.append((msg, display_name))
```

```
    # Post each message
    for msg, sender in conversation:
        text = msg.get('content', '').strip()
        timestamp = msg.get('originalarrivaltime', '')
        if timestamp:
            try:
                # Parse the ISO 8601 timestamp using datetime.fromisoformat
                timestamp = datetime.fromisoformat(timestamp.replace("Z", "+00:00")).strftime('%Y-%m-%d %H:%M:%S')
            except ValueError:
                print(f"❌ Failed to parse timestamp: {timestamp}")

        if msg.get('messagetype') == 'RichText':
            # Normal text message
            if text:
                payload = {
                    'channel_id': channel_id,
                    'message': f"[{timestamp}] {sender}: {text}"
                }
                response = requests.post(mattermost_post_url, headers={**headers, 'Content-Type': 'application/json'}, json=payload)
                if response.status_code != 201:
                    print(f"❌ Failed to send message: {response.status_code} {response.text}")
```

```
    elif msg.get('messagetype') == 'RichText/UriObject':
        # It's a media file
        match = re.search(r'doc_id="([^"]+)"', text)
        if match:
            doc_id = match.group(1)
            possible_files = [f for f in os.listdir(media_folder) if f.startswith(doc_id)]
            if possible_files:
                filepath = os.path.join(media_folder, possible_files[0])
                with open(filepath, 'rb') as f:
                    files = {'files': (os.path.basename(filepath), f)}
                    file_upload_payload = {
                        'channel_id': channel_id
                    }
                    upload_response = requests.post(mattermost_file_url, headers=headers, files=files,
data=file_upload_payload)
                    if upload_response.status_code == 201:
                        file_id = upload_response.json()['file_infos'][0]['id']
                        post_payload = {
                            'channel_id': channel_id,
                            'message': f"[{timestamp}] {sender} uploaded a file:",
                            'file_ids': [file_id]
                        }
```

```
                post_response = requests.post(mattermost_post_url, headers={**headers, 'Content-Type': 'application/json'}, json=post_payload)

                if post_response.status_code != 201:

                    print(f"❌ Failed to post uploaded file message: {post_response.status_code} {post_response.text}")

            else:

                print(f"❌ Failed to upload file: {upload_response.status_code} {upload_response.text}")

        else:

            print(f"⚠️ No local file found for doc_id {doc_id}")



    # Pause a bit to avoid API flooding

    time.sleep(0.2)



  # Post migration summary after each conversation

  post_migration_update(topic, len(members), len(conversation))
```

Finally, the “post_migration_update” function is ran to post an update about the recently migrated conversation into the “Migration Documentation” channel.

```
def post_migration_update(conversation_name, num_members, num_messages):

  message = f"The conversation '{conversation_name}' has been migrated. ({num_members} members, {num_messages} messages)."

  payload = {

    'channel_id': migration_channel_id,

    'message': message

  }
```

```
    response = requests.post(mattermost_post_url, headers={**headers, 'Content-Type':
'application/json'}, json=payload)

    if response.status_code != 201:

        print(f"❌ Failed to post migration update: {response.status_code} {response.text}")
```